

BigClaw AI — Design Basis and Description

Autonomous Investment Research & Portfolio Management System

Field	Detail
System Name	BigClaw AI
Document No.	DBD-AI-001 Rev. 1
Version	March 28, 2026
Author	Curtis Biggs
Hostname	BigClaw
Dashboard	https://bigclaw.grandpapa.net
GitHub Review Copy	https://github.com/cbiggs1-tech/bigclaw-codebase

Table of Contents

1. [System Description](#)
2. [Hardware Platform](#) - 2.1 Hardware Specifications - 2.2 Development Access
3. [Software Architecture](#) - 3.1 System Diagram - 3.2 Codebase - 3.3 Services
4. [Database Schema](#) - 4.1 Tables
5. [Portfolio Management Philosophy](#) - 5.1 Portfolio Definitions - 5.2 Portfolio Parameters
6. [Decision Engine & Scoring System](#) - 6.1 Signal Dimensions (20) - 6.2 Style-Specific Signal Weights - 6.3 Score-to-Label Mapping - 6.4 Special Rules - 6.5 Style Gate Checks - 6.6 AI Gate Reasoning - 6.7 Candidate Discovery Screener
7. [Autonomous Trading Logic & Execution](#) - 7.1 Execution Flow - 7.2 Position Sizing - 7.3 Safety Rules
8. [Risk Management — Trailing Stops](#) - 8.1 Trail Percentages by Portfolio - 8.2 Stop Lifecycle - 8.3 Intraday Stop Monitor
9. [Scheduled Operations & Automation](#) - 9.1 OpenClaw Cron Jobs (Weekday) - 9.2 OpenClaw Cron Jobs (Weekly) - 9.3 OpenClaw Cron Jobs (Daily) - 9.4 System Crontab Jobs - 9.5 Disabled Jobs
10. [Data Integrity & Anti-Hallucination Pipeline](#)
11. [Data Feeds & External APIs](#)
 - o 11.1 Market Data & Pricing

- 11.2 Options Flow & Institutional Intelligence
 - 11.3 Sentiment & Social
 - 11.4 News & Research
 - 11.5 ARK Invest Tracking
 - 11.6 Economic Calendar & Macro
 - 11.7 Weather & Environment
 - 11.8 Website Frontend
 - 11.9 LLM Providers
 - 11.10 Communication
 - 11.11 Technical Analysis Libraries
12. [User Interfaces](#)
- 12.1 Slack — Primary Interactive Channel
 - 12.2 GitHub Pages Dashboard
 - 12.3 Dashboard Data Files
13. [Logging, Monitoring & Security](#)
- 13.1 Application Logs
 - 13.2 Trade Logs
 - 13.3 Intraday Stop Check Logs
 - 13.4 API Retry
 - 13.5 Configuration Files
 - 13.6 Security
14. [Key Design Principles](#)
-

1. System Description

BigClaw AI is a fully autonomous investment research agent and paper-trading portfolio manager that runs 24/7 on a single Raspberry Pi 4 Model B. Its purpose is simple and unchanging: gather, synthesize, and analyze market data from every available source — price action, technicals, fundamentals, sentiment, insider activity, and macro signals — then act on that intelligence to manage seven distinct style-specific paper portfolios.

System Functions:

- **Research** — continuously monitors the market without human intervention.
- **Manage** — executes simulated trades inside strict, philosophy-driven rules using Alpaca's paper-trading API (each portfolio begins with \$100,000 virtual cash).
- **Protect** — enforces position limits, trailing stops, concentration caps, and a hard "no-trade" rule outside the 10:00 AM–4:00 PM ET weekday window.

BigClaw reports every morning (market overview), every afternoon (portfolio performance), and in real time (trade alerts) via Slack and the public dashboard at bigclaw.grandpapa.net. It improves itself through weekly research sessions, style-compliance audits, and decision-engine refinements.

Important reminder for anyone who inherits or touches this system: BigClaw is strictly paper trading. It has never and will never execute real-money trades. Any future attempt to connect it to a live brokerage account must be treated as a complete redesign.

2. Hardware Platform

BigClaw runs on a Raspberry Pi 4 Model B. This choice is deliberate: it is low-power, silent, always-on, and inexpensive to replace. The entire system — code, database, logs, and GitHub Pages refresh — lives on this one device. The local Windows copy and GitHub repository exist only as review snapshots; the Pi is the single source of truth.

Development happens primarily through VS Code Remote SSH (key-based ed25519 authentication). The OpenClaw agent runtime and Claude Code CLI are also installed directly on the Pi.

2.1 Hardware Specifications

Component	Specification
Board	Raspberry Pi 4 Model B Rev 1.4
Processor	Broadcom BCM2711, ARM Cortex-A72 (4 cores @ 1.8 GHz)
Architecture	aarch64 (64-bit ARM)
Memory	8 GB LPDDR4
Storage	256 GB USB SSD (228 GB usable, 8% utilized)
Operating System	Debian GNU/Linux 13 (Trixie)
Kernel	Linux 6.12.62+rpt-rpi-v8 SMP PREEMPT
Hostname	BigClaw
Network	Local LAN at 192.168.1.171, SSH alias <code>bigclaw</code>
Power	Continuous operation, auto-restart on failure

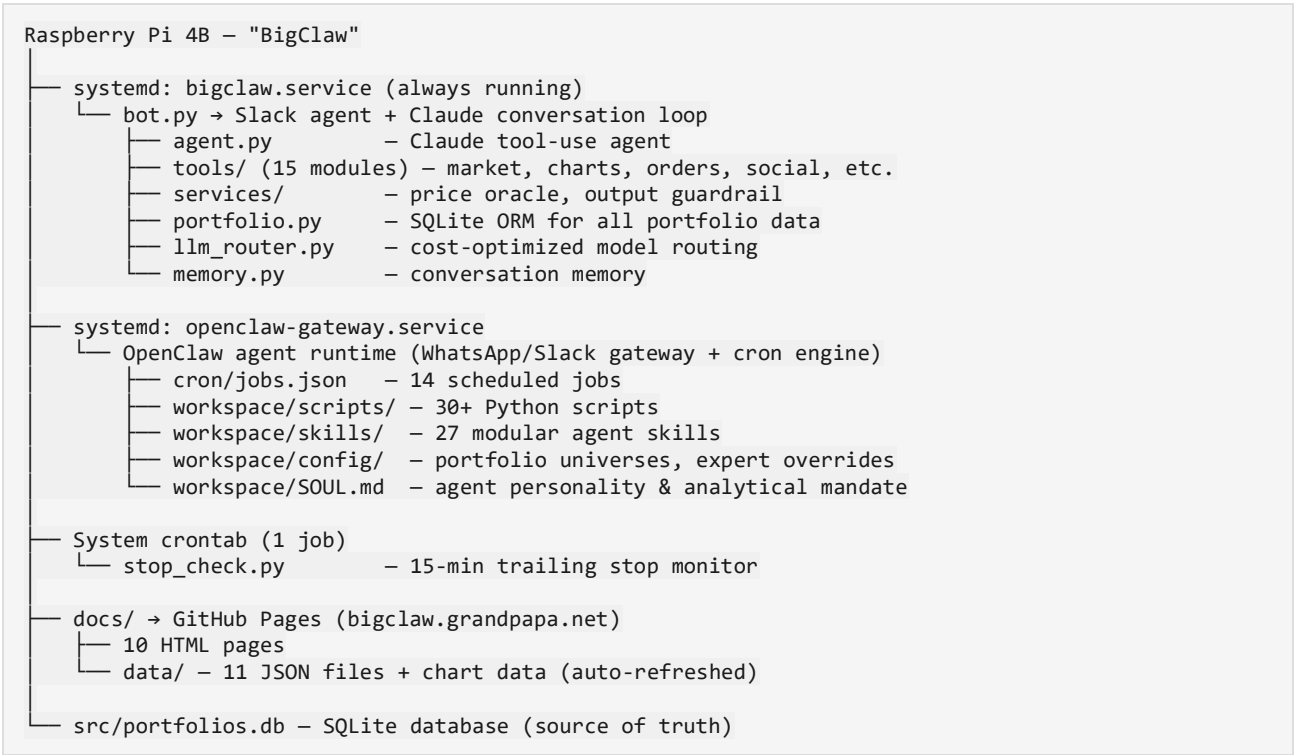
2.2 Development Access

Method	Detail
VS Code Remote SSH	Primary development interface from Windows desktop
Claude Code CLI	Installed on Pi via OpenClaw (~/.openclaw/)
SSH from Windows	ssh bigclaw (key-based auth, ed25519)

3. Software Architecture

The system is deliberately split into two cooperating codebases so that data collection, trading logic, and interactive conversation stay fast and cheap while heavy reasoning is offloaded to cloud LLMs only when needed.

3.1 System Diagram



3.2 Codebase

3.2.1 Core Application — ~/bigclaw-ai/

The core application is the always-on Slack bot, the portfolio database, and the public dashboard. It runs as `bigclaw.service` under `systemd` and is the user-facing side of BigClaw.

Slack Bot (`src/bot.py`) — Built on `slack-bolt` with Socket Mode (no inbound webhooks, no public ports). The bot listens for direct messages and channel mentions, routes each message through the Claude agent with full conversation memory, and returns the response. Image-based responses (charts) are uploaded as Slack files. The bot requires three tokens: `SLACK_BOT_TOKEN`, `SLACK_APP_TOKEN`, and `SLACK_SIGNING_SECRET`.

Claude Agent (`src/agent.py`) — Implements a tool-use conversation loop against the Anthropic API. When a user sends a message, the agent passes it to Claude along with the full set of available tools. If Claude invokes a tool, the agent executes it locally, feeds the result back, and continues the loop until Claude produces a final text response. Conversation history is maintained per-channel via `memory.py` so context carries across messages.

Interactive Tools (`src/tools/`) — 36 tools organized across 10 modules that Claude can invoke during conversation:

Module	Tools	What They Do
<code>market.py</code>	<code>get_stock_quote</code> , <code>get_stock_details</code>	Live price, fundamentals, P/E, market cap, sector
<code>charts.py</code>	<code>generate_stock_chart</code> , <code>compare_stocks_chart</code> , <code>generate_macd_chart</code> , <code>generate_rsi_chart</code> , <code>generate_bollinger_chart</code> , <code>generate_monte_carlo_chart</code> , <code>generate_moving_averages_chart</code>	Technical analysis chart generation (uploaded as images)
<code>news.py</code>	<code>get_yahoo_news</code> , <code>search_financial_news</code>	Per-ticker and broad financial news
<code>social.py</code>	<code>get_x_sentiment</code> , <code>get_stocktwits_sentiment</code> , <code>search_reddit_stocks</code> , <code>get_wsb_trending</code>	Social sentiment from X/Twitter, StockTwits, Reddit
<code>predictions.py</code>	<code>search_polymarket</code> , <code>get_polymarket_trending</code>	Prediction market odds
<code>portfolio.py</code>	<code>create_portfolio</code> , <code>delete_portfolio</code> , <code>list_portfolios</code> , <code>view_portfolio</code> , <code>buy_stock</code> , <code>sell_stock</code> , <code>get_transactions</code> , <code>compare_portfolios</code>	Full portfolio CRUD and trade execution
<code>orders.py</code>	<code>set_stop_loss</code> , <code>set_limit_buy</code> , <code>set_limit_sell</code> , <code>view_pending_orders</code> , <code>cancel_order</code>	Pending order management

Module	Tools	What They Do
<code>strategy_analyzer.py</code>	<code>run_analysis_now</code>	Trigger immediate decision engine analysis
<code>technical.py</code>	Additional technical analysis tools	Extended TA capabilities
<code>base.py</code>	<code>set_report_channel</code> , <code>set_autonomous_trading</code>	System configuration

LLM Router (`src/llm_router.py`) — A lightweight cost-optimization layer that routes LLM calls to the cheapest model capable of the task. Routine data summarization goes to Gemini Flash Lite via OpenRouter (~\$0.01/call). Analytical work requiring deeper reasoning is routed to Claude Sonnet (~\$0.20/call). Interactive Slack conversations use Claude Sonnet directly via the Anthropic API. The router exposes `call_openrouter()` for general calls and `summarize_with_flash()` as a convenience wrapper for cheap summarization tasks.

Services (`src/services/`): - **Price Oracle** (`price_oracle.py`) — Single source of truth for all prices system-wide. In-memory cache with 10-minute TTL. Refuses to serve stale data. - **Output Guardrail** (`output_guardrail.py`) — Scans every outbound message for `xxx.xx` price patterns and cross-checks against the oracle. Corrects any deviation > 2%.

Other components: - `src/portfolio.py` — SQLite ORM for all portfolio data (holdings, transactions, snapshots, orders, trailing stops) - `docs/` — GitHub Pages website with auto-refreshed JSON data files - `logs/` — Application logs (rotated at 5 MB, 2 backups) - `src/portfolios.db` — SQLite database (source of truth for all portfolio state)

3.2.2 Automation Layer — `~/.openclaw/workspace/`

The automation layer runs as `openclaw-gateway.service` under `systemd`. It provides the cron engine that drives all scheduled operations, 30+ standalone Python scripts for data gathering and trading, modular agent skills, and the `SOUL.md` personality file.

Scripts (`scripts/`) — 30+ standalone Python scripts that handle everything the LLM should not be trusted to do on its own: data gathering (`morning_data_gather.py`, `afternoon_data_gather.py`), price refresh and dashboard updates (`price_refresh.py`), autonomous trading (`autonomous_trader.py`), decision engine scoring (`decision_engine.py`, 20 signal dimensions), style gate enforcement (`style_gates.py`), AI gate reasoning (`gate_reasoning.py`), weekly candidate discovery (`candidate_screener.py`), style compliance auditing (`style_compliance.py`), trailing stop management (`trailing_stop_manager.py`, `stop_check.py`), ARK tracking (`ark_itk_tracker.py`), and portfolio reconciliation. All scripts source credentials from `~/.env_secrets` and log via `bigclaw_logging.py`.

Configuration (`config/`): - `portfolio_universes.json` — Per-portfolio allowed ticker lists (holdings + candidates) - `expert_overrides.json` — Manual conviction score overrides for specific tickers

SOUL.md — The agent's personality, analytical mandate, and behavioral rules. This file is loaded into every OpenClaw agent session and defines who BigClaw is. Key directives include:

- **Prime Directive:** Truth and accuracy above all. Protect the user's financial resources. Objectivity over narrative.
- **Analytical Mandate:** Lead with conviction and directional bias, then explain. Synthesize conflicting signals explicitly. Be bold and interpretive — state clear leans, scenarios with rationale, and thesis application. Minimize raw data dumps; prioritize interpretation.
- **Personality:** Sharp, direct, confident when warranted, brutally honest, forward-looking. Humor auto-adjusts by context (TARS-style calibration: low for alerts, higher for casual chat).
- **Anti-Hallucination:** Never invent prices, percentages, or news. Always call a data tool before responding to any market query. If tools fail, state "data unavailable" — never guess.
- **Autonomy Guardrail:** Never autonomously fix code, edit configs, or debug issues unless explicitly asked. Report problems and wait. This is a hard rule that overrides all other instructions.
- **Fact Verification:** Pull live data via `stock_quote.py` (primary), `technical_analysis.py` (technicals), or web search (fallback) before every analytical response. Cross-verify when possible; flag discrepancies > 2%.
- **Mandatory Skill Usage:** When asked to fact-check, must load and run the `truth-engine` skill (4 independent AI models through a SIFT framework).

Skills — 30 modular analytical capabilities the agent can load on demand. 27 are installed in `~/.openclaw/skills/` (from the ClawHub marketplace) and 3 are custom workspace skills:

Skill	Description
<code>backtest-expert</code>	Expert guidance for systematic backtesting — parameter robustness testing, slippage modeling, bias prevention, interpreting results
<code>finance-news</code>	Market news briefings with AI summaries across US/Europe/Japan markets
<code>financial-planning</code>	Solopreneur business finances — budgeting, cash flow, P&L, projections
<code>fundamental-stock-analysis</code>	Structured scoring playbook — quality, balance sheet, cash flow, valuation, peer ranking
<code>institutional-flow-tracker</code>	Track 13F filings — hedge fund and institutional ownership changes, smart money flow
<code>intellectia-stock-forecast</code>	AI-powered entry/exit analysis, target price predictions, probability calculations
<code>log-analyzer</code>	Parse, search, and analyze application logs; debug from log files; error pattern analysis

Skill	Description
market-environment-analysis	Comprehensive global market analysis — risk-on/off assessment, sector analysis, technical indicators
news-summary	Fetch and summarize news from trusted international RSS feeds (workspace custom)
onchain	Crypto portfolio tracking, market data, CEX history, transaction lookups
openinsider	SEC Form 4 insider trading data — director, CEO, and officer buys/sells
options-strategy-advisor	Options pricing (Black-Scholes), Greeks, strategy P&L simulation, earnings plays
portfolio-manager	Portfolio analysis via Alpaca — allocation, risk metrics, diversification, rebalancing
python-dataviz	Professional data visualization with matplotlib, seaborn, plotly — static and interactive
realtime-x-sentiment-tracker	Real-time X/Twitter sentiment polling every 2 hours, alerts on shifts > 5%
sector-analyst	Sector and industry performance analysis — market cycle assessment, rotation patterns
sec-watcher	Monitor SEC EDGAR filings for 50+ AI/tech companies — 8-K, 10-K, 10-Q, insider transactions
self-improve	Agent self-diagnosis — fix configs, prompts, response quality, personality tuning
skill-vetting	Security and utility assessment before installing third-party ClawHub skills
stock-evaluator	Comprehensive stock evaluation — valuation, fundamentals, technicals, entry prices, conviction ratings
stock-strategy-backtester	Backtest trading strategies on historical OHLCV data — win rate, CAGR, drawdown, Sharpe ratio
time-series-analysis	Time series feature engineering, model training, and forecasting
truth-engine	Multi-model fact-checking — 4 independent AI models through SIFT framework with consensus scoring (workspace custom)

Skill	Description
uptime-kuma	Interact with Uptime Kuma monitoring server — check status, add/remove monitors
us-market-bubble-detector	Quantitative bubble risk scoring via revised Minsky/Kindleberger framework — VIX, margin debt, breadth, IPO data
yahoo-finance	Stock prices, quotes, fundamentals, earnings, options, dividends, analyst ratings via yfinance
afrexai-esg-reporting	ESG scoring on Environmental, Social, and Governance factors aligned with 2026 disclosure standards
afrexai-portfolio-risk	Monte Carlo simulations, Value at Risk, stress tests, drawdown, beta, Sharpe ratio
afrexai-tax-planning	Tax-loss harvesting suggestions, entity structure optimization, deduction planning

3.3 Services

Two systemd services run permanently:

Service	Type	Purpose
bigclaw.service	systemd (always-on)	Slack bot + Claude agent + 36 interactive tools
openclaw-gateway	systemd (always-on)	Agent runtime, cron engine, Slack/WhatsApp gateway

A lightweight crontab job (`stop_check.py` every 15 min during market hours) runs outside the LLM loop for zero-token-cost monitoring.

4. Database Schema

The database (SQLite) is deliberately simple and self-contained. Everything that matters is here; nothing is hidden in external services. All portfolio state lives in `src/portfolios.db`.

All scripts connect with WAL (Write-Ahead Logging) journal mode and a 30-second busy timeout. WAL allows concurrent readers and writers without "database is locked" errors — critical because cron jobs (`stop_check`, `price_refresh`) overlap with the autonomous trader during market hours. Every `sqlite3.connect` call across all 15+ scripts enforces `PRAGMA journal_mode=WAL` and `PRAGMA busy_timeout=30000`.

4.1 Tables

```
portfolios
  id, name, investment_style, starting_cash, current_cash,
  report_channel, is_active, purchase_status, created_at

holdings
  id, portfolio_id, ticker, shares, avg_cost, rationale,
  first_bought_at, last_bought_at

transactions
  id, portfolio_id, ticker, action (buy/sell), shares, price,
  total_value, rationale, executed_at

daily_snapshots
  id, portfolio_id, snapshot_date, total_value, cash,
  holdings_value, daily_return

pending_orders
  id, portfolio_id, ticker, order_type (stop_loss/limit_buy/limit_sell),
  trigger_price, shares, status, triggered_at

trailing_stops
  id, portfolio_id, ticker, high_water_mark, hwm_date, trail_pct,
  trigger_price, status (active/triggered/stale), created_at,
  updated_at, triggered_at
```

5. Portfolio Management Philosophy

BigClaw manages nine independent paper portfolios. Seven are tied to distinct investment philosophies modeled after legendary investors or thematic approaches; the decision engine applies unique signal-weighting matrices to each style so that a ticker that scores as a "STRONG BUY" for Momentum Growth may legitimately score as a "HOLD" or "SELL" for Value Picks. This style fidelity is non-negotiable and is the single most important mechanism that prevents the system from drifting into generic "just buy what's hot" behavior. The eighth portfolio (added June 2026) is a controlled experiment in LLM-driven discretionary trading with no rule-based engine — see section 5.3.

5.1 Portfolio Definitions

#	Portfolio	Investment Style	Modeled After
1	Value Picks	Quality Value Investing	Buffett / Graham
2	Innovation Fund	Disruptive Innovation	Cathie Wood / ARK
3	Growth Value	Growth at Reasonable Price (GARP)	Peter Lynch

#	Portfolio	Investment Style	Modeled After
4	Income Dividends	Income / Dividend Growth	Dividend Aristocrats
5	Momentum Growth	CANSLIM Momentum	William O'Neil
6	Nuclear Renaissance	Nuclear Energy / Domain Expertise	Thematic Structural
7	AI Defense & Autonomous	AI Defense / Autonomous Systems	Pentagon Spending Theme
8	LLM-ETF Focus	LLM-Driven Autonomous, ETF-tilted (3-Sonnet dialectic — see 5.3)	None — pure model judgment
9	LLM-Comando	LLM-Driven Single-Stock (3-Sonnet dialectic, stock-preference enforced — see 5.3)	None — pure model judgment

5.2 Portfolio Parameters

Parameter	Value
Starting capital per portfolio	\$100,000 virtual cash

5.3 LLM Discretionary Portfolio (experimental, June 2026 onward)

The 8th and 9th portfolios are deliberate counter-designs to the seven rule-based portfolios. They share the same 3-Sonnet dialectic architecture but differ in candidate universe and prompt preference:

- **LLM-ETF Focus** (formerly named LLM Discretionary, renamed June 10 2026) — original variant, trades whatever the dialectic settles on with no preference. Observed behavior is sector-rotation via SPDR ETFs as the LLM defaults to the most data-rich investable objects (XLK, XLF, XLE, etc.) and rotates between them on macro/news shifts. Curtis observed this and decided to preserve the behavior intact as a real-world test of LLM-driven sector rotation.
- **LLM-Comando** (new June 10 2026) — variant with **strict stock-preference language** in all three agent prompts (Bull, Bear, Judge). ETFs are explicitly forbidden except for hedging. Candidate universe expanded to include ~60 curated liquid names from `portfolio_universes.json` (the BigClaw rule-based portfolios' shared universe) so per-ticker Benzinga news flows into the context for individual-stock theses. Dry-run verified: Judge picked AMAT (analyst PT raises) and APA (geopolitical energy thesis) — no ETFs.

The original architecture description below applies to both variants. Differences noted inline.

Common architecture: No style gates, no top-10 rotation, no target-price discipline, no decision-engine scoring. Every trade decision is made by a 3-Sonnet dialectical decision system.

Architecture: 3-Sonnet dialectic. Three Claude Sonnet 4.6 agents run sequentially each weekday at 11:00 CT: 1. **BULL agent** — given the data (portfolio state, sector/factor ETFs, news, journal), builds the strongest case FOR candidate trades. 2. **BEAR agent** — reads the bull's case and same data, builds the strongest case AGAINST. Stage 1 fact-verifies every Bull claim against the data feed; Stage 2 then runs a **mandatory ALREADY-PRICED-IN test** on each surviving thesis — freshness (when did the catalyst go public?), price reaction (has the stock already moved on it? if so, entering is chasing the reaction), and durability (durable or reflexive driver?). A true-but-already-priced thesis is a disqualifying weakness, not a tradeable one (strengthened June 15 2026 after the LLM-Comando AAL miss — bought a day-old oil/MOU catalyst already in the price). 3. **JUDGE agent** — reads data + bull + bear + journal. Its **first move is gap-analysis, not adjudication**: before weighing Bull vs Bear it must name what is absent from BOTH cases that would change the decision (is the catalyst already priced in? durable or reflexive driver? what would make this wrong that neither side named?), emitted in a required `gap_analysis` field. This is the alpha-generating function of the more capable Judge seat (Comando runs Opus 4.8 here vs Sonnet on ETF Focus — an active A/B) — synthesizing two advocates only averages views already in the price; the edge is in the shared omission (added June 15 2026). It must then explicitly address the strongest bear counter-arguments before committing. Produces strict-JSON trade decisions with required `exit_thesis` field (specific gain target / stop loss / time-based exit).

Recursive learning via journal. Every cycle appends to `data/llm_journal.jsonl` — input snapshot, bull case, bear case, judge decision, executed trades, and (filled in later by Python reconciler) realized P&L vs the predicted `exit_thesis`. Each subsequent cycle reads this journal as input, so the model can see its own track record of wins, losses, and prediction accuracy. Over weeks, the model's stated `patterns_noted` field accumulates as a self-written strategy guide.

Anti-cheating constraints (mechanical, enforced in Python): - Ticker validation: every proposed trade ticker is verified against Alpaca's tradable assets list before submit. - Cash wall: cannot spend more than `current_cash`; enforced by `record_trade`. - Market-hours gate: trades only submitted when Alpaca clock reports market open. - Hallucination prevention via prompt: training cutoff is January 2026; every factual claim must cite the data feed; no inventing news, earnings, or events.

Safety rail — catastrophic drawdown freeze. If total portfolio value drops below \$50,000 (50% of starting capital), `LLM_PORTFOLIO_DRAWDOWN_FREEZE.flag` is written, trading pauses, and a Slack alert fires. Manual resolution required to resume.

Cost. LLM-ETF Focus daily cycle ≈ \$0.13 (three Sonnet 4.6 calls, smaller news context). LLM-Comando daily cycle ≈ \$0.18 (same architecture but expanded ~60-ticker candidate universe inflates input tokens). Combined annual cost: ≈ \$80 for ~252 trading days × 2 portfolios.

Trade closure reconciler (added June 8 2026). A second script `llm_portfolio_reconciler.py` runs daily at 15:30 CT (30 min after market close) to verify each open position against its stated `exit_thesis`. Logic: in priority order, check `stop > target > time_exit`; if any trigger fires, submit the SELL via the canonical Alpaca + `record_trade` path. The Judge prompt now emits a structured `exit_conditions: {target_pct, stop_pct, time_exit_date}` alongside the prose `exit_thesis`; for legacy trades lacking the structured field, a small Sonnet 4.6 call (~\$0.001) extracts the conditions on first encounter. Each closure writes an `outcome` record to `data/llm_outcomes.jsonl` capturing entry/exit prices, realized %, days held, which trigger fired,

and whether the original prediction was correct. **This outcomes log is what makes recursive learning real** — without verified prediction outcomes, the daily Judge has no signal to learn from. Periodic synthesis (monthly, to be built ~July 1) will compress accumulated outcomes into a `llm_strategy_guide.md` that the daily prompt reads at top-of-context.

Intraday trigger watcher (added June 8 2026). Per Curtis: "11 AM was my preconceived bias. The LLM should trade when IT sees opportunity." The morning cycle was moved from 11:00 to 09:00 CT, and the JUDGE prompt now requires an `intraday_triggers` field where the LLM specifies up to 8 watch conditions per day — price-based (e.g., "if NVDA breaks \$210"), news-based (e.g., keyword list like ["Fed", "Powell", "FOMC", "rate decision"]), or time-based (e.g., final-hour check). A new script `scripts/llm_portfolio_watcher.py` runs every 5 minutes during market hours (cron `* /5 8-15 * * 1-5`), reads the morning's pending triggers from `data/llm_pending_triggers.json`, polls yfinance for price triggers, fetches CNBC RSS + Reuters via Google News for news triggers, and checks time triggers against the clock. When any trigger matches, a focused single-Sonnet LLM call (no Bull/Bear dialectic — the thesis was already formed this morning) decides: execute the original `action_intent` as planned, modify it, or stand down. Decisions execute via the canonical Alpaca + `record_trade` pipeline. Daily budget cap: max 6 fires per day across all triggers, tracked in the state file's `fires_today` counter. Each fire costs ~\$0.05; worst case daily total is \$0.13 morning + $6 \times \$0.05 = \0.43 (\$110/year). Each fire appends a `trigger_response` entry to the journal and an "Intraday Trigger Fire" section to today's `data/llm_decisions/YYYY-MM-DD.md`, so Curtis can browse exactly what triggered, how the LLM responded, and what executed.

Experimental purpose. The 7 rule-based portfolios are the control. This portfolio is the treatment. Direct comparison over months: does LLM judgment with structured self-feedback beat hand-coded rules and SPY? Either outcome is valuable — it informs the future direction of BigClaw's decision architecture. | Maximum holdings per portfolio | 10 | | Minimum holdings per portfolio | 7 (triggers swap/add if breached) | | Maximum single position | 20% of total portfolio value (holdings + cash) | | Rebalance trim target | 18% of total portfolio value | | Concentration monitoring | Every 2 hours (Slack warning if > 20%) | | Auto-rebalance | First trading day of each month | | Cash reserve | Minimum 2% of starting capital retained | | Maximum single order | \$25,000 |

Parameter constraints and expert overrides are defined in configuration files so future maintainers can expand or tighten them without touching core code.

Each portfolio operates within a curated universe of allowed tickers stored in `config/portfolio_universes.json`. The universe is split into two tiers: **holdings** (tickers the portfolio currently owns) and **candidates** (pre-approved tickers eligible for swap-in when the decision engine identifies a stronger alternative). Specific tickers change over time as the engine recommends swaps and the market evolves — the configuration file is the authoritative, living source for current membership.

Value Picks — The universe centers on blue-chip stalwarts that Benjamin Graham would recognize: mega-cap consumer staples, financials with fortress balance sheets, healthcare giants, and legacy industrials trading below intrinsic value. These are companies with decades of earnings history, wide moats, and the kind of boring predictability that deep-value investors prize. The decision engine heavily weights P/E, debt-to-equity, and insider activity for this portfolio.

Innovation Fund — This is the widest universe, deliberately so. It spans disruptive technology across AI, quantum computing, biotech, fintech, cloud infrastructure, and high-growth SaaS — companies that are redefining industries or creating entirely new ones. Many of these names are volatile and unprofitable by

traditional metrics, which is exactly why the engine zeroes out P/E and debt signals for this style and instead emphasizes revenue growth, relative strength, and momentum crossovers.

Growth Value — A tightly curated universe of large-cap compounders that deliver strong earnings growth at reasonable valuations — the "best of both worlds" names that Peter Lynch built his career on. These are household-name technology and financial companies that dominate their markets but still have room to grow. The engine balances technical and fundamental signals roughly equally for this style.

Income Dividends — REITs, utilities, MLPs, consumer staples, and dividend aristocrats — companies selected primarily for reliable, growing income streams. The universe favors names with long dividend track records, sustainable payout ratios, and defensive characteristics during market downturns. Bond market signals and dividend yield carry double weight in this portfolio's scoring.

Momentum Growth — The universe is sector-agnostic and changes the most frequently. It tracks names showing the strongest relative strength, price acceleration, and technical breakouts — wherever they occur. The engine applies its heaviest momentum weighting here (MACD, SMA crossovers, and relative strength all at 2×) and largely ignores fundamentals. This portfolio has the tightest trailing stop (10%) because momentum names reverse fast.

Nuclear Renaissance — A focused thematic universe covering the nuclear energy supply chain: uranium miners, fuel processors, reactor designers (both conventional and SMR), and nuclear-adjacent power generators. This is a structural thesis — the multi-decade buildout of nuclear capacity for AI data centers and grid decarbonization — so the universe is deliberately small and deep rather than broad.

AI Defense & Autonomous — Defense primes, autonomous systems developers, drone manufacturers, space companies, and cybersecurity firms positioned to capture the Pentagon's accelerating spending on AI-enabled warfare and autonomous platforms. The universe includes both established contractors and emerging pure-plays. Revenue growth carries double weight here, reflecting the sector's rapid budget expansion.

Universe files can be updated without touching core code. The decision engine's `--rescreen` mode evaluates all candidates in each universe and recommends swaps when a candidate outscores the weakest current holding by 3 or more points.

Treasury Reserve (retired May 1, 2026). An eighth portfolio held \$300K of SGOV as a money-market reserve. It was retired after per-portfolio cash walls (full reset April 10) made a separate cash-bucket portfolio unnecessary, and after Alpaca paper trading was confirmed not to credit dividends — making the SGOV strategy unable to validate in paper mode. The portfolio row remains in the DB with `is_active=0` for historical preservation; no transactions are accepted while inactive. All website and report logic filters by `is_active=1` so retired portfolios are invisible.

6. Decision Engine & Scoring System

The analytical heart of BigClaw is `decision_engine.py`. It evaluates every candidate ticker across 20 signal dimensions (technical, fundamental, quality, sentiment, macro, and override factors) and then multiplies

those raw signals by a style-specific weight matrix (0 = ignore, 1 = normal, 2 = double emphasis). The result is a composite score that is mapped to clear action labels.

Before a ticker even reaches the scoring engine, it must pass through **style gate checks** (see Section 6.5) — hard pre-buy filters that ensure only eligible stocks enter a portfolio's scoring pipeline. Gates are the bouncer; weights are the judge.

This weighted, multi-factor approach is why BigClaw stays true to each portfolio's thesis instead of chasing the same momentum names across every style.

6.1 Signal Dimensions

#	Category	What It Measures	Score Range
1	RSI	Relative Strength Index (14-day)	-1 to +1
2	MACD	MACD/signal line crossovers	-0.5 to +1
3	SMA50	Price vs 50-day simple moving average	-0.5 to +1
4	SMA200	Price vs 200-day simple moving average	-0.5 to +1
5	Cross	Golden cross (50 > 200) or death cross	-0.5 to +1
6	RelStrength	1-month return vs sector ETF	-0.5 to +1
7	EarningsGrowth	Quarterly earnings growth rate	-1 to +1
8	RevenueGrowth	Revenue growth rate	-1 to 0
9	PE	Forward P/E vs trailing P/E direction	-1 to 0
10	DebtEquity	Debt-to-equity ratio (flag if > 100)	-1 to 0
11	ShortInterest	Short float percentage (flag if > 10%)	-1 to 0
12	Insider	Net insider buying vs selling (20 transactions)	-1 to +1
13	BondMkt	Bond market signal (fed funds, yield curve)	Variable

#	Category	What It Measures	Score Range
14	ValueOverride	Analyst target discount, RSI oversold, P/B < 1, capitulation volume, expert overrides	0 to +5
15	DividendYield	Dividend yield level	-1 to +1
16	PEG	P/E ÷ earnings growth (Lynch's key metric). Uses best of forward/trailing PEG.	-1 to +1
17	ROE	Return on equity (Buffett's #1). ≥20% excellent, ≥15% good, <10% weak.	-1 to +1
18	FCF	Free cash flow positive/negative + FCF yield context if >5%.	-1 to +1
19	GrossMargin	Gross margin width: ≥40% wide moat, ≥30% decent, <30% thin.	-1 to +1
20	PayoutSafety	Dividend payout ratio: <60% safe, <80% moderate, >80% stretched. Skips non-dividend stocks.	-1 to +1

Signals 1–6 are **Technical**, 7–10 are **Fundamental**, 16–20 are **Quality Fundamentals** (new), and 11–15 are **Sentiment / Macro / Override**.

6.2 Style-Specific Signal Weights

Each portfolio multiplies signal categories differently. Weight 0 = ignore, 1 = normal, 2 = double emphasis. The 5 new quality dimensions (PEG, ROE, FCF, GrossMargin, PayoutSafety) allow portfolios to differentiate on fundamental quality rather than relying solely on price-action and binary earnings signals.

Signal	Value	Innovation	Growth	Income	Momentum	Nuclear	Defense
RSI	0.5	0.5	0.5	0.5	1.5	1	1
MACD	0	1	0.5	0	2	1	1
SMA50	0	1	0.5	0.5	2	1	1
SMA200	0.5	0.5	1	0.5	1.5	1	1
Cross	0	1	0.5	0	2	1	1

Signal	Value	Innovation	Growth	Income	Momentum	Nuclear	Defense
RelStrength	0	1.5	1	0	2	1.5	1.5
EarningsGrowth	1	0.5	1	1	1.5	1	1
RevenueGrowth	0.5	2	1	0.5	0.5	1.5	2
PE	1.5	0	0.5	1	0	0.5	0.5
DebtEquity	1.5	0.5	1	1.5	0	1	1
ShortInterest	0.5	0.5	1	0.5	1	1	1
Insider	1.5	0.5	1.5	1	0.5	1.5	1.5
BondMkt	0.5	0.5	0.5	2	0	0.5	0.5
ValueOverride	2	0	1	1	0	1	0.5
DividendYield	1	0	0.5	2	0	0	0
PEG	0.5	0	2	0	0	0	0
ROE	2	0	1	0.5	1	0.5	0.5
FCF	2	0	1	1.5	0	1	0.5
GrossMargin	1.5	0	0.5	0	0	0	0
PayoutSafety	0	0	0	2	0	0	0

Key design choices in the weight matrix: - **Value Picks**: ROE=2, FCF=2, GrossMargin=1.5 — quality is the defining characteristic. - **Growth Value**: PEG=2 — Lynch's single most important metric, replaces separate PE + EarningsGrowth dependency. - **Income Dividends**: PayoutSafety=2, BondMkt=2, DividendYield=2 — sustainable income over yield-chasing. - **Momentum Growth**: EarningsGrowth=1.5 — O'Neil requires earnings-confirmed momentum, not pure price. - **Innovation Fund**: All quality signals zeroed — irrelevant for pre-revenue/high-growth disruptors.

6.3 Score-to-Label Mapping

Score	Label	Action
≥ 3	STRONG BUY / ADD	Buy candidate
1 to 2	BUY / HOLD	Hold or add on dip
0	HOLD	No action
-1 to -2	WATCH / CAUTION	Monitor closely
≤ -3	SELL / TRIM	Trim or exit

6.4 Special Rules

Special rules baked into the engine enforce discipline:

- **Bond headwind override:** If bond market signal ≤ -2 and ticker score is marginal (1-2) with weak value override, score is forced to 0 (no buy).
- **Value override cap:** If raw technical score ≤ -5 and value override is < 3 , the override is prevented from rescuing the ticker above -3 (still flagged as sell/trim).
- **Rescreen mode:** When run with `--rescreen`, the engine also scores candidate tickers from portfolio universes and generates swap recommendations (candidate must outscore weakest holding by ≥ 3 points).

6.5 Style Gate Checks (`style_gates.py`)

Style gates are hard pre-buy filters that run **before** the scoring engine. A ticker that fails a gate is blocked from the portfolio regardless of how high it might score. This is the primary mechanism preventing portfolio convergence — it ensures a pure AI company never enters Nuclear Renaissance, and a zero-dividend growth stock never enters Income Dividends.

Three severity levels:

Severity	Meaning	AI Reasoning?
pass	Allowed into scoring pipeline	No
gate	Borderline — close to failing	Yes (escalated to Opus)
reject	Hard block, no exceptions	No

Per-portfolio gate rules:

Portfolio	Gate Checks
Value Picks	P/E < 35, ROE > 10%, positive earnings. Hard reject: P/E > 60.
Growth Value	PEG < 3.0 (best of forward/trailing). Hard reject: P/E > 80.
Income Dividends	Dividend yield ≥ 1.5%. REITs/MLPs/ETFs exempt from payout check.
Innovation Fund	Must connect to 1 of 5 platforms (AI, genomics, fintech, energy, space). Legacy industries rejected.
Momentum Growth	Earnings growth, ROE, institutional ownership gates (O'Neil CANSLIM).
Nuclear Renaissance	Sector gate: only nuclear-related tickers. Blocks 82% of raw utility screens.
AI Defense & Autonomous	Sector gate: defense-related only. Blocks 69% of raw tech screens.

Entity-type awareness: REITs, MLPs, and ETFs are exempt from payout ratio checks. Banks are exempt from FCF checks. Companies with negative book equity (aggressive buybacks like LOW) are exempt from ROE checks.

6.6 IPS Criteria Are Authoritative (`gate_reasoning` removed May 2026)

Style gates are pure deterministic rules from each portfolio's IPS, defined by the April 2026 four-LLM thesis debate. A ticker either passes a gate or fails it; there is no AI override layer.

A previous design had a `gate_reasoning.py` module that escalated borderline failures to Claude Opus 4.6 for a BLOCK/ALLOW decision. That module was retired May 1, 2026 and moved to `scripts/attic/`. Reasons: - It generated ~470 Opus calls per Saturday (~\$28/run, ~\$1,460/year) for what amounted to second-guessing the IPS design - The 20-dimension scoring engine already handles dynamic re-evaluation — when a held stock's metrics deteriorate, its score drops and best-in-class rotation kicks it out - Design principle: if a stock fails today's gate, it fails. If its metrics improve later, it passes the next screen and is re-evaluated for entry. No need for an AI to second-guess each case.

The four-LLM debate produced explicit criteria for each portfolio (Value Picks: $P/E \leq 25 + ROE \geq 15\%$; Income Dividends: 10-year streak + Chowder Rule + yield $\geq 1.5\%$; Nuclear and AI Defense use whitelists). Those criteria are the design. Anything that fails them is rejected; anything that passes is scored by the 20-dim engine and considered for entry by best-in-class rotation.

6.7 Candidate Discovery Screener (`candidate_screener.py`)

The candidate screener is a weekly pipeline that discovers new stocks for each portfolio. It runs every Saturday at 9:00 AM CT and follows this flow:

1. **Screen** — Finviz screener with portfolio-specific filters (2–3 filter sets per portfolio)
2. **Gate** — Deterministic IPS-criteria gate checks filter raw results. Failures are dropped — no AI override.
3. **Update** — Survivors are added to `portfolio_universes.json` as candidates (max 10 per portfolio, matching the holding cap)
4. **Compare** — The daily rescreen (`decision_engine --rescreen`) scores new candidates against holdings
5. **Swap** — If a candidate outscores the weakest holding by ≥ 3 points (`SWAP_THRESHOLD`), the swap fires: sell incumbent, buy challenger

This closes the loop on candidate freshness — the portfolio universes are no longer static hand-curated lists but are refreshed weekly with market-wide discovery.

7. Autonomous Trading Logic & Execution

Trading is orchestrated once per day at 10:30 AM ET by `autonomous_trader.py` (and reinforced by the intraday stop-check script). The flow is intentionally sell-first: liquidate weak positions and triggered stops before deploying cash into new opportunities. This prevents over-allocation and ensures cash is always available for the best opportunities.

7.1 Execution Flow

1. **Sync** — Reconcile SQLite DB with Alpaca paper account (positions + cash)
2. **Analyze** — Run `decision_engine.py --json --rescreen` to generate signals
3. **Monthly rebalance** — On the first trading day of each month, trim any position exceeding 20% of portfolio value back to 18%
4. **Sells first** — Free cash before buying: - Score ≤ -5 : SELL ALL (100% of position) - Score ≤ -3 : TRIM (50% of position) - Swap recommendation: SELL ALL (replace with stronger candidate)
5. **Trailing stop check** — Execute any triggered trailing stops (between sells and buys)
6. **Buys** — Deploy available cash: - **Style gate check first** — every buy candidate must pass `passes_style_gate()` before execution. Blocked tickers are logged and skipped. - Score ≥ 3 required to buy (normal mode) - Score ≥ 2 if portfolio is $> 60\%$ cash (deployment mode) - Score ≥ 1 if part of a swap recommendation
7. **Post-trade sync** — Re-compare DB positions against Alpaca after all trades complete. Only mismatches that persist after execution are reported. This eliminates false alerts from pre-trade discrepancies that the trades themselves resolve.
8. **Summary** — Post execution report to Slack

7.2 Position Sizing

Score	Allocation	Typical Size
>= 5	12% of starting capital	~\$12,000
3 to 4	8% of starting capital	~\$8,000
1 to 2 (swap/deploy)	5% of starting capital	~\$5,000

7.3 Safety Rules

These rules must never be relaxed:

- **Trading window:** Weekdays, 10:00 AM – 4:00 PM ET only. Enforced at cron level, script level, and Alpaca execution level.
- **Cash reserve:** Minimum 2% of starting capital retained per portfolio
- **Max single order:** \$25,000
- **Round-trip limit:** Max 3 round-trips per ticker per week (prevents churning)
- **Max holdings:** 10 positions per portfolio
- **Min holdings:** 7 positions per portfolio (triggers swap/add when below)
- **Position concentration:** No single position may exceed 20% of total portfolio value (holdings + cash)

8. Risk Management — Trailing Stops

Every held position automatically receives a style-specific trailing stop. The stop only ever moves upward (high-water-mark ratcheting) — never down. `stop_check.py` runs every 15 minutes during market hours with a single `yfinance` batch call (near-zero cost). Under extreme stress a one-line `tighten_stops(factor)` call can halve all trail percentages instantly. This combination of ratcheting logic + frequent lightweight checks is one of the most important capital-preservation mechanisms in the entire system.

The architecture has three defensive layers, hardened in May 2026 after a bug-class audit found 22 of 60 held positions had broken (stale or triggered) trailing stops:

Layer 1 — Code prevention. Every sell path (autonomous_trader rotation, trailing-stop trigger, `trade_executor` manual) DELETes the `trailing_stops` row when the position fully closes. New buys create a fresh active stop via `initialize_stops` which DELETes any leftover row first, then INSERTs (replacing the prior INSERT OR IGNORE that silently dropped new stops when stale rows already existed for the ticker).

Layer 2 — Invariant assertion. `stop_check.py` runs `assert_invariants()` after each maintenance pass. If any held non-SGOV position lacks an active stop, OR any active stop exists for a ticker no longer held, a Slack DM alert fires loudly. Surveillance catches anything Layer 1 misses.

Layer 3 — Self-heal. `remove_stale_stops()` actually DELETes orphan rows (was previously just marking them `status='stale'` and leaving them in place — that buggy pattern blocked new INSERTs and is what produced the original 22 broken stops). The `autonomous_trader` also calls `initialize_stops` at the end of its run so positions bought at 10:00 CT don't sit unprotected for up to 15 minutes waiting for the next `stop_check` fire.

The same orphan invariant for `holdings` is checked by `data_health_check.py` every 2 hours and at end of each `autonomous_trader` run.

8.1 Trail Percentages by Portfolio

Portfolio	Trail %	Rationale
Value Picks	18%	Trust the thesis, wide band
Innovation Fund	18%	Volatile by nature
Income Dividends	15%	Moderate protection
Growth Value	12%	GARP discipline
Nuclear Renaissance	12%	Thematic structural
AI Defense & Autonomous	12%	Thematic structural
Momentum Growth	10%	Cut losers fastest

8.2 Stop Lifecycle

1. **Initialize:** When a new holding is added (or buy completes via `autonomous_trader`), `initialize_stops` DELETes any leftover row for the ticker and INSERTs a fresh active stop with $HWM = \max(\text{current_price}, \text{avg_cost})$. This always produces a clean active row even if a stale entry existed from a prior position.
2. **Ratchet:** Every 2 hours (price refresh) and every 15 minutes (stop check), if $\text{price} > HWM$, the HWM moves up and the trigger price moves up proportionally.
3. **Trigger:** If $\text{price} \leq \text{trigger_price}$, position is sold at market and the `trailing_stops` row is DELETED in the same transaction.
4. **Tighten:** During market stress, `tighten_stops(factor)` cuts all trail bands (e.g., $\text{factor}=0.5$ cuts 18% to 9%). Circuit-breaker mechanism.
5. **Cleanup:** When a position is sold by any path (rotation, trim, manual, trigger), the matching `trailing_stops` row is DELETED in the same DB transaction. `remove_stale_stops()` also runs each cycle and DELETes any orphan rows.

8.3 Intraday Stop Monitor (`stop_check.py`)

Lightweight Python script that runs every 15 minutes during market hours via system crontab. Zero LLM token cost.

- Fetches prices for all held tickers in one `yf.download` batch call (~3 seconds)
 - Runs full stop cycle: `remove_stale_stops` (DELETes orphans), `initialize_stops` (creates new), `ratchet_stops` (move HWMs up), `assert_invariants` (Slack alerts on coverage gap), `check_triggers` (fire stops)
 - If a stop triggers: executes Alpaca market sell, updates DB (DELETes holdings + DELETes `trailing_stops` + INSERTs sell transaction), posts to Slack
 - Enforces same 10 AM – 4 PM ET trading window as all other components
-

Stop-Sell Cooldown (Anti-Round-Trip)

When a trailing stop fires, the IPS scoring engine often still flags the same ticker as top-10 minutes later, leading to immediate rebuy at near-identical price. The May 1 2026 AI Defense incident — LMT/NOC/RTX sold and re-bought within 73 minutes for ~\$4.9K of crystallized losses — exposed this.

The `stop_cooldowns` table records every (portfolio, ticker) that exits via trailing stop, along with the score-at-trigger from the latest `signals.json`. The buy gate in `autonomous_trader._execute_buy_order`'s caller blocks rebuys until **BOTH**:

1. **At least 10 days have elapsed** since the stop trigger.
2. **The current score is at least 2 points above the score at trigger.**

If score never recovers, the position stays blocked indefinitely — the right behavior for a genuinely deteriorated name. The cooldown row is auto-deleted the moment both conditions clear, so the gate is single-firing.

The gate sits at the rotation buy site only — it does not block sells, manual `trade_executor` buys, or rebalance trims. Implementation: `scripts/stop_cooldown.py`.

Order Fill Discipline

`scripts/order_fill.py` is the single source of truth for waiting on Alpaca order fills. The May 6 2026 ANET incident exposed the bug class: a sell order for 73 shares filled fully on Alpaca, but `trailing_stop_manager.py` checked the order one second later, saw 37 of 73 filled (status `partially_filled`), substring-matched on "filled," and recorded only 37 shares to the DB. Alpaca filled the remaining 36 a moment later — DB never updated, \$5,281 of cash drift.

`wait_for_fill()` enforces a strict contract: **the returned (`filled_qty`, `filled_price`) match Alpaca's terminal state for the order, period.** Implementation:

- Polls every 2s up to `timeout_s` (default 30s) for terminal status (`filled` / `canceled` / `expired` / `rejected` / `done_for_day`).
- On primary timeout, requests cancel of the open remainder.
- Polls every 0.5s for up to 10s after cancel (cancel is async; the open quantity can race-fill or race-cancel — both produce a deterministic terminal state).
- Returns the filled quantity at the terminal moment. No "Alpaca fills more shares later that the DB doesn't see" drift is possible.

All five fill-recording paths use this module: `autonomous_trader._execute_sell_order`, `autonomous_trader._execute_buy_order`, the REBALANCE-SELL block, the in-trader STOP-SELL block, `trailing_stop_manager.execute_stop_sells`, and `trade_executor.place_order`. Zero local copies of fill-wait logic remain.

Screening Method — Consistent Application

The screening pipeline has exactly three stages and N-selection happens only at the third:

1. **Finviz query** (`candidate_screener.py`) — broad sector/cap/valuation filter producing raw candidates.
2. **IPS gates** (`style_gates.py`) — hard per-portfolio filters (Buffett value, Wood innovation, O'Neil momentum, etc.). Every ticker that passes a portfolio's gates is a legitimate candidate for that portfolio.
3. **Decision-engine scoring + top-N selection** — every gate-passing ticker is scored on the 20-dimension scoring model. The trader holds the top 10 by score; everything else is a candidate the model can rotate into.

Hard rule: stage 2's output is the universe. There is no cap, no truncation, no secondary filter between stages 2 and 3. Selection by score happens once, in stage 3.

The May 8 2026 alphabetic-bias incident was caused by a hidden stage 2.5: a `MAX_CANDIDATES_PER_PORTFOLIO = 20` cap combined with `sorted()` iteration, which silently kept only the alphabetically-first 20 of however many gate-passers existed. Innovation Fund's 158 gate-passers were reduced to 20 starting A or B; 138 valid C-Z candidates were never seen by the scoring engine. Same bug bit Income Dividends, Value Picks, and Growth Value with varying severity.

The fix removes the cap entirely. Decision-engine compute scales linearly with universe size; current measurement: ~1.4s per ticker, ~400-500 unique tickers across 7 portfolios after the fix → ~10-12 minute decision-engine runs (vs ~4 min before). The cron schedule has 60-150 minute gaps between runs — comfortably accommodates the slower runs.

If gate-pass counts grow beyond what compute permits, the answer is parallelization or batching (Finviz scraping is the bottleneck), not reintroducing a screener cap. **A cap that drops gate-passers by criteria other than the gate criteria is by definition inconsistent screening.**

Accounting Integrity — Three Invariants

The simplified accounting model: **transactions are the single source of truth**. Per-portfolio cash and holdings are recomputable views of the transaction log, not independently mutated state. The audit verifies this with three invariants that must always hold:

Invariant 1 — Per-portfolio cash double-entry

$$\text{starting_cash} - \Sigma(\text{buys.total_value}) + \Sigma(\text{sell.s.total_value}) = \text{current_cash}$$

Tolerance: \$0.01. Failure means the shadow ledger has drifted from its own transaction log (a write went wrong).

Invariant 2 — DB shares match Alpaca shares (per ticker)

$$\Sigma(\text{holdings.shares}) \text{ for ticker } T = \text{Alpaca position qty for } T$$

Tolerance: 0 shares. Failure means a buy or sell did not record the right qty (partial-fill bug, lost write, or double-write).

Invariant 3 — Aggregate cash matches Alpaca account cash

$$\Sigma(\text{per-portfolio current_cash}) \approx \text{Alpaca.account.cash} - \text{unclaimed_setup_cash}$$

Tolerance: \$5 noise. The Alpaca paper account may have pre-existing setup cash that no portfolio claims. Any unexplained gap beyond setup cash means real money has gone unaccounted.

`scripts/accounting_audit.py` runs all three daily at 06:15 CT (before `daily_export`). Failure writes `logs/ALPACA_MISMATCH.flag` and posts a CRITICAL Slack alert. Pass posts a green check daily so you can see the audit ran.

Why this matters: in May 2026 the trailing-stop partial-fill bug silently siphoned \$5,281 from Momentum Growth (recording 37 shares sold when Alpaca filled 73). The existing reconciliation only checked share counts (Invariant 2) — both DB and Alpaca read 0 shares post-sell, the share check passed, the cash discrepancy went undetected. Invariant 1 + 3 together would have caught it the next day.

The May 11 2026 reset (`scripts/full_reset_monday.py`) liquidates all positions, wipes the shadow ledger, and re-establishes each portfolio at \$100K starting cash. Both ledgers start equal; the audit verifies they stay equal.

Dashboard same-day-buy bug (May 26 2026 — fixed): the dashboard's per-portfolio `dailyReturn` field was credited with market moves that happened BEFORE we owned a position. For each holding, the dashboard computed `prev_value = shares * prev_close` (yesterday's close). For positions opened on the current day, `prev_close` is a price from before the position existed, so the daily-return calc captured gains the portfolio never actually realized. Surface symptom: after a 6-position rotation today, Momentum

Growth showed +4.32% daily gain while the broader market was flat. MU alone contributed +\$1,620 of phantom gain because it gapped from \$751 to \$875 over the Memorial Day weekend before BigClaw bought it. Fix: when `first_bought_at` equals today's date, use `avg_cost` (the fill price) as `prev_price`. New positions then contribute only their real intraday move since fill. After the fix, MG's daily return resolved to a realistic +0.65%. Prior-day positions are unaffected — their `prev_close` falls within the holding period.

IV tracker — 30-day data-gathering experiment (started May 31 2026): `scripts/iv_tracker.py` runs daily at 11:00 CT via system crontab. Captures forward-looking options-implied volatility signatures (skew = OTM-put-IV minus ATM-call-IV; spread = ATM-call-IV minus ATM-put-IV; classification = BULLISH/MIXED/BEARISH) at ~30d and ~60d expiries for all holdings plus the top-10 scored candidates per portfolio (~80-130 unique tickers/day). Data lands in the `iv_history` SQLite table. Motivation: a literature review of UW/options-flow predictive signals identified IV skew and call-put IV spread as the only signals with documented forward-equity-return edge for our long-only days-to-months horizon — and they're derivable free from yfinance, no UW subscription required. A one-shot diagnostic across current holdings showed +1.10% mean excess vs SPY for BULLISH-IV names vs -1.94% for BEARISH (~3% spread). This experiment gathers enough point-in-time data to validate or refute that spread across ~20 trading days and ~80 tickers, then inform a clean decision-engine integration (or not) after the validation window. Does NOT modify the decision engine. Pure collection.

SHOP partial-trim bug (May 21 2026 — fixed): a SAFETY TRIM 50% on SHOP sold 57 of 114 shares correctly at Alpaca, but the DB deleted the entire holdings row. Root cause: every sell call site computed `sell_all = (filled_qty >= requested_sell_qty)`, which is True for any partial trim where Alpaca fills the full requested amount. For TRIMs the requested qty is half the position, so the row got deleted even though half the shares remained. Fix: `trade_recorder.record_trade` now ignores the `sell_all` parameter and always decrements with cleanup-when-zero. Computing remaining shares from the DB itself is the only reliable source of truth; caller-supplied flags were a footgun. Caught by Invariant 2 (DB shares vs Alpaca shares) the next audit cycle.

Target-Price Capture at Entry

Every new position records `target_price` on the holdings row at entry time. Source: yfinance `info["targetMeanPrice"]` — analyst consensus 12-month target. Also recorded: `target_set_at` (timestamp) and `target_source` ("yfinance_mean").

Semantics: - **Captured on first buy** of a position (when no holdings row exists yet) - **Preserved on subsequent adds** — adding to the position does not overwrite the original target - **Cleared when position fully closes** (holdings row deleted)

The target is captured now (schema migration 2026-05-11) so future Phase 2 work can activate a target-price hold discipline without retrofitting historical data. Phase 2 will define rules like "hold until `current_price >= target_price * 0.90`, sell only on thesis break before then, or trim if position > 15% of portfolio."

Phase 2 is not yet active — daily-rotation discipline remains the rule. But every position bought from 2026-05-11 forward has the target persisted, so when Phase 2 flips on we have the data we need.

Entry-Timing Signals (May 14 2026)

Five signal categories that shift the scoring engine from late-cycle momentum confirmation toward earlier-stage entry detection. Goal: stop the engine from buying stocks at all-time highs where the score is high precisely because the move is already over.

Extension penalties (penalize buying at top-of-rally): - `Extension90d`: penalty when stock is up >15% in last 90 days (graduated: -0.5 to -2) - `ExtensionMA`: penalty when stock is >15% above 200-day MA (graduated: -1 to -2)

Breakout rewards (reward buying at start-of-rally): - `Breakout`: reward when price freshly punches above 60-day high (+1 to +2) - `VolContract`: reward when 30-day volatility is in bottom quartile vs 1-year (+1 to +2) - `MARecLaim`: reward when stock freshly reclaims 50-day MA after being below (+2)

Style-tuned weights (in `STYLE_WEIGHTS`): - Innovation Fund / Value Picks / Growth Value / Income Dividends: extension 1.5×, breakout 1.5× - Momentum Growth: extension 0.5× (riding momentum is the strategy), breakout 2.0× (catch early) - Nuclear / AI Defense: 1.0× across all

Validation status: backtested against 5 pre-reset winners and 6 recent losers. Breakout signals correctly boosted DDOG/ARGX/AMBA at their entry points. Extension penalty correctly flagged ANAB at +57% as extended (though ANAB continued running — false positive on this case). The recent losers (HUBS/PODD/RBLX) were already falling at entry — different bug class (falling-knife detection) not addressed by these signals. That work belongs to a future phase.

Research Dossier Pipeline (Phase A)

`scripts/research_dossier.py` generates one-page Markdown research dossiers for candidate tickers. Pulls data from all available sources (yfinance info, eps_trend, analyst revisions, signals.json, options_flow.json, holdings, target_price) and uses Claude Sonnet 4.6 to synthesize into a structured doc.

Output format: - Signal snapshot (BigClaw score, analyst consensus, EPS revisions, valuation) - Bull case (3-4 specific points with data citations) - Bear case (3-4 specific points) - Key unknowns (questions a domain expert should ask) - One-line recommendation

Output location: `docs/dossiers/{ticker}_{YYYYMMDD}.md`

Cost: ~\$0.02-0.03 per dossier (Sonnet input + output). 70 dossiers/week = \$1-2/week or ~\$78/year — comfortable within R&D budget.

Usage: - Single ticker: `python3 scripts/research_dossier.py --ticker AMD` - Whole portfolio: `python3 scripts/research_dossier.py --portfolio "Innovation Fund"` - All top-10: `python3 scripts/research_dossier.py --top10` - Data dump only: `python3 scripts/research_dossier.py --ticker AMD --dry-run`

Phase A (manual invocation only) ships May 15. Phase B (Slack review UI) and Phase C (trader gates on Curtis approval) deferred until Phase A is used in practice for 2-4 weeks to validate the format and information density.

9. Scheduled Operations & Automation

All recurring work is defined in OpenClaw's `cron/jobs.json`. Disabled jobs are explicitly listed so no one accidentally re-enables them.

9.1 OpenClaw Cron Jobs (Weekday)

All times Eastern unless noted.

Time	Job	Model	Purpose
8:55 AM	Morning Data Gather	Gemini Flash Lite	Runs <code>morning_data_gather.py</code> , writes raw data to <code>/tmp/bigclaw_morning_data.txt</code>
9:00 AM	Morning Market Analysis	Claude Sonnet	Reads morning data file, produces market analysis + portfolio implications, posts to Slack
~~9:00, 11:00, 1:00, 3:00 PM~~	~~Price Refresh (2hr)~~	—	DISABLED May 28 2026. Was an agent-turn cron that ran <code>price_refresh.py</code> via the LLM exec tool; broke when the OpenClaw exec-approval socket stopped being connected (every run pinged Curtis for approval). Redundant with <code>refresh_all.sh</code> (system cron, runs <code>price_refresh.py</code> 5x/day directly). Price refresh now runs solely via the native bash cron.
10:30 AM	Daily Autonomous Trading	Gemini Flash Lite	Runs <code>autonomous_trader.py</code> — full decision engine + trade execution cycle
4:25 PM	Afternoon Data Gather	Gemini Flash Lite	Runs <code>afternoon_data_gather.py</code> , writes raw data to <code>/tmp/bigclaw_afternoon_data.txt</code>
4:30 PM	Afternoon Portfolio Report	Gemini Flash Lite	Reads afternoon data file, produces portfolio performance report, posts to Slack

9.2 OpenClaw Cron Jobs (Weekly)

Schedule	Job	Model	Purpose
Saturday 8:00 AM CT	ARK ITK Summary	Gemini Flash Lite	Runs <code>ark_itk_tracker.py</code> , tracks ARK Invest transactions
Saturday 8:00 AM CT	Weekly Research Session	Claude Sonnet	Autonomous deep research into market themes and opportunities
Saturday 9:00 AM CT	Candidate Screener	Python (zero LLM) + Opus 4.6 (borderline)	Discovers new candidate stocks via Finviz, filters through style gates + AI reasoning, updates <code>portfolio_universes.json</code>
Saturday 9:00 AM CT	Weekly Style Compliance Audit	Gemini Flash Lite + Opus 4.6 (borderline)	Audits holdings against style gates with AI reasoning for borderline cases
Sunday 7:00 AM CT	Weekly Network Security Scan	Gemini Flash Lite	Runs security scan of the Pi
Sunday 9:00 AM CT	Weekly OpenClaw Version Check	Gemini Flash Lite	Checks for OpenClaw updates and model version changes

9.3 OpenClaw Cron Jobs (Daily)

Time	Job	Model	Purpose
7:00 AM CT daily	Good Morning + Security Check	Gemini Flash Lite	System health check, security scan

9.4 System Crontab Jobs

These jobs run via the Pi's system crontab — pure Python, zero LLM cost.

Schedule	Script	Purpose
7:30 AM CT, weekdays	<code>daily_export.sh</code>	Pre-market full export: sector heatmap, calendar, trades (last 2 days), analysis, news, performance chart, all per-ticker charts. Alerts Slack on failure.
9:00, 10:00, 12:00, 2:00, 4:30 PM CT, weekdays	<code>refresh_all.sh</code>	Signals + prices + charts refresh: runs decision engine with rescreen (<code>export_signals.py</code>), macro scanner, price refresh (<code>price_refresh.py</code>), and per-ticker chart export (<code>export_charts.py</code>). Enriches each signal with

Schedule	Script	Purpose
		portfolio holding context and swap recommendations. Commits and pushes to GitHub Pages. Alerts Slack on failure.
9:30 AM CT (10:30 AM ET), weekdays	<code>autonomous_trader.py</code>	Autonomous trade execution: syncs DB with Alpaca, runs decision engine with rescreen, executes sells/swaps/buys, post-trade sync check. The 9:00 AM signal refresh provides a 30-minute supervisory window before trades execute.
Every 15 min, 9 AM–3 PM CT, weekdays	<code>stop_check.py</code>	Intraday trailing stop monitor (zero LLM cost)

9.5 Disabled Jobs

Job	Reason
Tuesday Trading Execution	Replaced by Daily Autonomous Trading
Good Morning Curtis	Replaced by Good Morning + Security Check
Email Check: <code>fixit@grandpapa.net</code>	Paused

10. Data Integrity & Anti-Hallucination Pipeline

LLMs are never allowed to fabricate numbers. Every market report follows a rigid two-step process:

Step 1 — Data Gather (Python, No LLM)

Pure-Python data-gather scripts (`morning_data_gather.py`, `afternoon_data_gather.py`) pull every price, fundamental, and statistic and write them to flat text files with clear `=== SECTION ===` delimiters. Every number, price, and percentage is captured from live APIs before any LLM is involved.

Step 2 — Analysis (LLM Reads File Only)

The LLM is instructed to read only that file. The prompt enforces critical rules:

- Every number must appear verbatim in the data file
- No web search, no guessing, no fabrication
- If data is missing, state "data unavailable"

Runtime Guardrails

Two runtime guardrails reinforce this:

- **Price Oracle** (`src/services/price_oracle.py`): Single source of truth for all prices system-wide. In-memory cache with 10-minute TTL. Refuses to serve stale data.
- **Output Guardrail** (`src/services/output_guardrail.py`): Scans every outbound message for `$xxx.xx` patterns and corrects any deviation > 2% against the oracle.

This pipeline is the reason BigClaw's reports remain trustworthy even when LLMs are involved.

Trade Recording Integrity

All sell paths (`autonomous_trader` rotation, trailing-stop trigger, `trade_executor` manual) wait for Alpaca fill confirmation, then record the actual `filled_avg_price` into the `transactions` table — not the originally submitted limit price. This was hardened May 2026 after a cash-drift audit found ~\$47K of accumulated misrecording from older trades that had been logged at limit price; limit orders fill at-or-below limit, so historical trades systematically over-recorded cash spent.

Historical drift is bounded (no new contributions). Per-portfolio cash accounting (`starting_cash - SUM(buys) + SUM(sells) + SUM(dividends)`) is correct going forward — each portfolio sees only its own trades, and the cash wall isolation ensures one portfolio cannot spend another's cash.

LLM Token Surveillance

`scripts/llm_token_report.py` runs daily at 6 AM CT, aggregates the previous day's calls from `logs/llm_calls.jsonl`, and posts a Slack DM with per-script breakdown. Alerts loudly if any single script exceeds 1M tokens or \$5 in one day. Catches the pattern of a heavy LLM consumer creeping into the codebase undetected — the way `gate_reasoning` did before it was found burning ~\$1,460/year.

Symbol Format at the Alpaca Boundary

BigClaw stores tickers using the Yahoo/Finviz convention with hyphens for class-share suffixes (`BRK-B`, `BF-A`, `BF-B`, `MOG-A`). Alpaca uses dots (`BRK.B`, `BF.A`). `scripts/alpaca_symbols.py` provides `to_alpaca()` and `from_alpaca()` for translation at the API boundary — applied at every `submit_order` site (`autonomous_trader`, `trade_executor`, `trailing_stop_manager`, `bigclaw_full_reset`) and at `get_all_positions()` reads (so reconciliation compares apples to apples). The DB and the rest of the codebase only ever see the hyphen format. Added May 4 2026 after Income Dividends BUY orders for `BF-A` and `BF-B` were rejected by Alpaca with `asset not found`.

11. Data Feeds & External APIs

11.1 Market Data & Pricing

Source	Library/API	Data Provided
Yahoo Finance	yfinance 1.1.0	Price history, fundamentals, earnings dates, dividends, options chain, per-ticker news
Finviz	finvizfinance 1.3.0	Short interest, insider transactions, analyst ratings, price targets
Alpaca	alpaca-py 0.43.2	Paper trade execution, account state, position sync, real-time/extended-hours quotes
FRED	CSV API (free)	Michigan Consumer Sentiment, Personal Saving Rate, Treasury yield curve, credit spreads

11.2 Options Flow & Institutional Intelligence

Source	Method	Data Provided

Unusual Whales subscription cancelled May 31 2026. A deep-research pass over the academic literature found that UW's flagship signals (flow, dark pool, GEX, market tide, congressional trades) have no documented forward-equity-return edge for our long-only days-to-months horizon, and a point-in-time backtest on our own buys agreed (no positive spread between bullish-UW and bearish-UW picks). The one signal with documented edge (IV skew + call-put IV spread) is derivable for free from yfinance option chains and is now collected daily by iv_tracker.py. The decision engine never used UW data, so trade execution is unaffected. Scripts removed: options_intelligence.py, unusual_whales.py, uw_api_extended.py, tsla_watchdog.py. Data gathers morning_data_gather.py and afternoon_data_gather.py had their UW sections removed; the rest of their data continues to feed the Slack briefings.

11.3 Sentiment & Social

Source	Method	Data Provided
X/Twitter	Apify scraper API	Real-time ticker sentiment from financial Twitter
StockTwits	Apify scraper (fallback)	Retail sentiment per ticker
Reddit (WallStreetBets)	Direct API	Trending tickers, retail sentiment

Source	Method	Data Provided
CNN Fear & Greed Index	JSON endpoint (free)	Composite market sentiment score (0-100), used as contrarian indicator

11.4 News & Research

Source	Method	Data Provided
Motley Fool	RSS feed (feedparser)	Investing and market news via four RSS feeds
Yahoo Finance News	yfinance	Per-ticker headlines pulled alongside price data
Brave Search	REST API (API key)	On-demand web search for fact-checking, catalyst verification, and research queries
SEC EDGAR	REST API (edgartools)	Insider trades (Form 4), annual reports (10-K), regulatory filings
Polymarket	Gamma API (free)	Prediction market odds on economic/political events

11.5 ARK Invest Tracking

Source	Method	Data Provided
CathieArk.com	Web scrape	Daily ARK fund trade notifications — buys, sells, position changes across ARKK, ARKG, ARKW
ARKFunds.io	REST API (free)	Structured ETF trade data from ARK's daily disclosures (primary source)
YouTube / ARK ITK	YouTube Data API + youtube-transcript-api	Weekly "In the Know" video — auto-discovered, transcribed, and summarized

ARK tracking feeds the Innovation Fund portfolio directly and is summarized weekly via the Saturday ARK ITK cron job.

11.6 Economic Calendar & Macro

Source	Method	Data Provided
Federal Reserve	FOMC calendar scrape	Interest rate decision dates and policy meeting schedule
Bureau of Labor Statistics	Schedule scrape	CPI inflation and Nonfarm Payrolls release dates
Bureau of Economic Analysis	Schedule scrape	GDP release schedule — quarterly growth data
Yahoo Finance (Earnings)	yfinance	Next earnings dates and EPS estimates for portfolio holdings

11.7 Weather & Environment

Source	Method	Data Provided
Open-Meteo	REST API (free, no auth)	Daily weather forecasts for Alvarado, TX — included in morning briefings

11.8 Website Frontend

Technology	Purpose
Chart.js	Portfolio performance line charts (CDN via jsDelivr)
TradingView	Embedded symbol overview widget for live ticker lookup
Google Fonts (Inter)	Typography across all dashboard pages
GitHub Pages	Static site hosting — files pushed from Pi via git, zero infrastructure cost

11.9 LLM Providers

Provider	Models Used	Purpose	Cost Model
Anthropic (direct)	Claude Opus 4.6	Interactive development via Claude Code; style gate AI reasoning for borderline decisions	Pro subscription + API (~\$20–30/yr for gates)
Anthropic (direct)	Claude Sonnet	Interactive Slack conversations	Pro subscription

Provider	Models Used	Purpose	Cost Model
OpenRouter	Claude Sonnet 4.6	Morning analysis, weekly research (high reasoning)	~\$0.20/call
OpenRouter	Gemini 3.1 Flash Lite	Data gather, price refresh, trading (low cost)	~\$0.01/call
OpenRouter	Gemini 2.5/2.0 Flash Lite	Fallback models	~\$0.005/call

Fallback chain: Gemini 3.1 Flash Lite → Gemini 2.5 Flash Lite → Gemini 2.0 Flash Lite

11.10 Communication

Service	Purpose
Slack (Bot Token + App Token)	Primary interface — receives reports, trade notifications, interactive queries
Discord (Webhook)	Error alerts, synchronized reports

11.11 Technical Analysis Libraries

Library	Version	Purpose
ta	0.11.0	RSI, MACD, SMA, Bollinger Bands, technical indicators
pandas-ta	0.4.71b	Extended technical analysis
pandas	3.0.0	Data processing, time series
numpy	2.2.6	Numerical computation
matplotlib	3.10.8	Chart generation (performance, MACD, RSI, Bollinger, Monte Carlo)
ffn	1.1.2	Financial analysis (Sharpe ratio, max drawdown, risk metrics)

The complete, browsable list of all data sources with links and descriptions is maintained on the dashboard's Sources page at bigclaw.grandpapa.net/sources.html.

12. User Interfaces

12.1 Slack — Primary Interactive Channel

The Slack bot runs as `bigclaw.service` and exposes 36 interactive tools that the Claude agent can invoke during conversation. Users interact via direct message or channel mention; the agent maintains per-channel conversation memory. The full tool inventory is documented in Section 3.2.1.

12.2 GitHub Pages Dashboard

Hosted on GitHub Pages at **bigclaw.grandpapa.net** behind Cloudflare Access. Data is refreshed on two schedules:

- **5× daily** (9 AM, 10 AM, 12 PM, 2 PM, 4:30 PM CT) via `refresh_all.sh`: signals, prices, macro, charts
- **1× daily pre-market** (7:30 AM CT) via `daily_export.sh`: sector heatmap, calendar, trades, analysis, news, performance chart

Both scripts alert Slack on failure so stale data is never silent.

Page	File	Purpose
Home	<code>index.html</code>	Market overview, portfolio summary cards
Dashboard	<code>dashboard.html</code>	Today's Planned Actions (sells and strong buys only), portfolio cards, sector rotation heatmap, upcoming events, recent trades, market analysis
Portfolio	<code>portfolio.html</code>	Individual portfolio deep dive — holdings table, metrics, P&L
Signals	<code>signals.html</code>	Supervisory display and single source of truth for all signal categories: action required, watch/caution, opportunities, hold. Each signal shows portfolio holding tags (shares, position %, capacity status) and pending swap recommendations with score differentials. Also: macro overview, sentiment, bond market, alerts & warnings
Ticker	<code>ticker.html</code>	Individual ticker detail with candlestick, MACD, RSI, Monte Carlo charts + signal context
Chart Detail	<code>chart-detail.html</code>	Portfolio performance comparison chart
Sources	<code>sources.html</code>	Data source documentation, cron schedule reference

Page	File	Purpose
Privacy	privacy.html	Privacy policy
Nav	nav.html	Shared navigation component
Styles	styles.html	Shared CSS

Design rule: The dashboard shows only actionable items (planned trades). All detailed signal analysis lives on the signals page. This avoids redundancy and ensures the signals page is authoritative. The signals page serves as the supervisory display — like Tesla FSD (Supervised), the autonomous trader makes all decisions but the human sees the same data and reasoning, with a 30-minute window before trade execution to intervene if needed.

12.3 Dashboard Data Files

All JSON data in `docs/data/` is auto-refreshed and pushed to GitHub Pages:

File	Contents	Refresh
signals.json	Decision engine output with rescreen: scores, labels, reasons, portfolio holding context (shares, position %, capacity), swap recommendations with score differentials	5×/day
portfolios.json	All 8 portfolios with holdings, current prices, returns	5×/day
llm_portfolio.json	LLM Discretionary daily bull/bear/judge output + trades	1×/day weekday
llm_outcomes.jsonl	Append-only trade closure log: entry/exit, realized %, trigger, prediction accuracy	continuous
llm_pending_triggers.json	Today's armed intraday triggers + fires_today counter	per-cycle, per-fire
llm_comando_portfolio.json	LLM-Comando daily bull/bear/judge output + trades	1×/day weekday
llm_comando_journal.jsonl	LLM-Comando append-only journal	per-cycle
llm_comando_outcomes.jsonl	LLM-Comando trade closure log	continuous

File	Contents	Refresh
llm_comando_pending_triggers.json	LLM-Comando intraday triggers + fires counter	per-cycle, per-fire
sector_rotation.json	Weekly sector rotation report (LLM-synthesized)	1×/week Sunday
market.json	S&P 500, Dow, NASDAQ, VIX + sector rotation heatmap (11 GICS sectors)	5×/day (indices), 1×/day (sectors)
macro.json	Macro indicators, bond yields, sentiment, sector performance, VIX, verdict	5×/day
charts/*.json	Per-ticker OHLCV, MACD, RSI, Monte Carlo for ~50 held tickers	5×/day
trades.json	Recent trade history (last 2 trading days)	1×/day
calendar.json	Upcoming economic events (FOMC, CPI, NFP)	1×/day
analysis.json	Macro market scanner report (markdown)	1×/day
news.json	Financial news headlines (Motley Fool RSS)	1×/day
metadata.json	Last update timestamps, export status	Every refresh
performance_chart.png	Portfolio performance vs S&P 500 (SPY), rebased to Apr 16 2026 refactor baseline	1×/day

Note: `price_refresh.py` preserves sector data in `market.json` when refreshing index prices — it merges rather than overwrites.

13. Logging, Monitoring & Security

Everything is logged. ERROR/CRITICAL entries are mirrored to Discord. API calls use automatic retry logic. Secrets live only in `~/.env_secrets` (never in git). SSH is key-only, `fail2ban` is active, unattended upgrades run, and a weekly security scan is part of the Sunday routine. The GitHub repo is intentionally a read-only review copy — no secrets ever leave the Pi.

13.1 Application Logs

All scripts log via `bigclaw_logging.py`:

- **Log file:** `~/bigclaw-ai/logs/bigclaw.log` (rotated at 5 MB, 2 backups)
- **Format:** `YYYY-MM-DD HH:MM:SS [bigclaw.module_name] LEVEL: message`
- **Discord alerts:** ERROR and CRITICAL level messages are auto-posted to Discord webhook

13.2 Trade Logs

- **File:** `~/openclaw/workspace/logs/trades.log`
- **Format:** `YYYY-MM-DD HH:MM ET | ACTION | portfolio | ticker | details`
- **Actions logged:** BUY, SELL, STOP-SELL, REBALANCE-SELL, SWAP, TRIM, SKIP, ERROR

13.3 Intraday Stop Check Logs

- **File:** `~/bigclaw-ai/logs/stop_check.log`
- **Frequency:** Every 15 minutes during market hours

13.4 API Retry

All external API calls use `bigclaw_retry.py` for automatic retries on transient failures (configurable attempts, delay, and label for logging).

13.5 Configuration Files

File	Purpose
<code>~/env_secrets</code>	All API keys: Alpaca, Anthropic, Slack, Discord, X/Twitter (Apify), OpenRouter
<code>~/openclaw/openclaw.json</code>	Agent runtime config: models, channels, gateway settings
<code>~/openclaw/cron/jobs.json</code>	All 14 scheduled job definitions with schedules, models, prompts
<code>~/openclaw/workspace/config/portfolio_universes.json</code>	Per-portfolio allowed ticker lists (holdings + candidates), refreshed weekly by candidate screener
<code>~/openclaw/workspace/config/expert_overrides.json</code>	Manual conviction score overrides for specific tickers

File	Purpose
<code>~/openclaw/workspace/SOUL.md</code>	Agent personality, analytical mandate, behavioral rules
<code>/etc/systemd/system/bigclaw.service</code>	Systemd service definition for the Slack bot
<code>~/bigclaw-ai/.env</code>	Application environment variables (Slack tokens, API keys)
<code>~/bigclaw-ai/.gitignore</code>	Ensures secrets, databases, and caches are never committed

13.6 Security

- All API keys stored in `~/env_secrets` (not in git, not in any tracked file)
- `.env` and `*.db` files are in `.gitignore`
- SSH access via ed25519 key authentication only
- Weekly automated network security scan (Sunday 7 AM)
- Daily security check as part of morning routine
- `fail2ban` running for brute-force protection
- Unattended upgrades enabled for security patches
- GitHub codebase repo is a read-only review copy — no secrets ever pushed

14. Key Design Principles (Guidance for Anyone Who Comes After)

These nine principles are the operating manual. Violate them at the risk of breaking the system's integrity:

1. **The Pi is the source of truth.** All execution, state, and decisions happen on the Raspberry Pi. Local copies and GitHub are snapshots only.
2. **Zero-token where possible.** Data gathering, price refreshes, trailing stop checks, and trade execution are pure Python — no LLM tokens consumed. LLMs are reserved for synthesis, analysis, and interactive conversation.
3. **Style fidelity above all.** Each portfolio's signal weights are sacred. A stock that's a BUY for Momentum may be a HOLD for Value. Do not apply a generic scoring model across styles.
4. **Consistent trading window.** No trades before 10:00 AM ET or after 4:00 PM ET, weekdays only. Every component must enforce this — at cron level, script level, and Alpaca execution level.
5. **Anti-hallucination by design.** Data files first, LLM second. The 2-step pipeline (Python data gather → LLM analysis of flat files) prevents fabricated numbers. The Price Oracle and Output Guardrail are non-optional.

6. **Sell before buy.** The autonomous trader always executes sells first to free cash, then buys. This prevents over-allocation and ensures cash is available for the best opportunities.
7. **Stops only move up.** Trailing stop high water marks ratchet upward on price increases but never decrease. This is the primary capital-protection mechanism. This locks in gains progressively.
8. **Log everything.** Every trade, every API call, every decision is logged with timestamps and rationale. Post-mortem analysis depends on it.
9. **WAL mode everywhere.** Every SQLite connection uses WAL journal mode and a 30-second busy timeout. Multiple cron jobs overlap during market hours (stop checks, price refreshes, trading). WAL prevents database-locked errors that caused 14 failed trade recordings on 2026-03-30.

This document, the `ARCHITECTURE.md` in the repo, the `SOUL.md`, and the live Pi itself are the complete specification. If you are reading this because you are taking over maintenance, start by running the weekly style-compliance audit and the security scan, then read the logs. The system is designed to be understandable, auditable, and safe — as long as these principles are respected.

Document generated March 30, 2026. Based on live system state from Raspberry Pi "BigClaw" at 192.168.1.171.